МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»
Центр образовательных программ топ-уровня в сфере
информационных технологий Образовательного центра Института №8

ЛАБОРАТОРНАЯ РАБОТА

по дисциплине «Операционные системы и архитектура компьютера»

на тему: **«Системные вызовы»**

Выполнил:

студент гр. М8О-206БВ-25

Толстопятов Трофим Михайлович

Проверил: Ефимов Д.А.

Оценка:

Дата:

Москва 2026

СОДЕРЖАНИЕ

Цель работы 2
Среда и методика эксперимента 3
Условия наблюдения 3
Регистрация и группировка событий 3
Сводные результаты 4
Анализ операций cscript 5
Запуск cscript и загрузка движка 5
Создание и запись файла 7
Чтение файла 9
Удаление файла 9
Запуск дочернего процесса cmd 10
Закрытие cscript 12
Сопоставление наблюдений и неожиданные результаты 14
Вывод 16
Приложения 17
Приложение А: скрипт test.vbs 17
Приложение Б: файлы с логами ProcMon 17

Цель работы

Цель работы - исследовать обращения программы к ресурсам Windows при выполнении пользовательских действий и связать наблюдаемые операции с функциями программы.

Объектом исследования выбран командный хост сценариев cscript.exe (Windows Script Host, консольный вариант) при выполнении скрипта на VBScript. Работа cscript состоит из двух принципиально разных этапов: сначала хост по расширению файла определяет язык, находит и загружает соответствующий движок (vbscript.dll), а затем уже движок исполняет код, обращаясь через COM к библиотекам scrrun.dll (FileSystemObject) и wshom.ocx (WScript.Shell). Именно эта двухступенчатая схема «хост + движок + COM-объекты» делает cscript интересным объектом: в одной трассе видны и инициализация процесса, и динамическая загрузка библиотек, и файловые операции, и создание дочернего процесса.

Исследуемый скрипт выполняет четыре действия: создаёт файл cscript_test.txt во временном каталоге и записывает в него строку; читает этот файл; удаляет файл; запускает дочерний процесс cmd /c echo Son proccess и дожидается его завершения. Набор подобран так, чтобы охватить принципиально разные механизмы: инициализацию процесса и поиск движка через реестр, динамическую загрузку DLL, чтение/запись/удаление файла и создание дочернего процесса.

Среда и методика эксперимента

Условия наблюдения

Параметр	Значение
Операционная система	Windows 11, версия 26H2
Процессор	Intel Core i5 12500H
Оперативная память	16 ГБ
Средство диагностики	Microsoft Sysinternals Process Monitor v4.11
Исследуемая программа	cscript.exe (C:\Windows\System32\cscript.exe)

Регистрация и группировка событий

Перед запуском скрипта буфер захвата очищался, после чего включалась запись событий. Сразу после появления строки End. и завершения процесса захват останавливался, а трасса сохранялась в формате CSV и PML. Применялся фильтр Process Name is cscript.exe с действием Include.
Это свойство фильтра напрямую определило границы наблюдения: события дочернего процесса cmd.exe (PID 27944) в трассу не попадают, виден только сам факт его создания операцией Process Create. Количественные сводки получены штатным средством ProcMon Tools → Count Occurrences по колонке Operation.

Сводные результаты

В таблице приведено число зарегистрированных событий по группам

Действие	Всего	Файлы	Реестр	Процессы
Запуск cscript и загрузка движка	2650	324	2170	156
Создание и запись файла	479	70	384	25
Чтение файла	3	3	0	0
Удаление файла	6	5	0	1
Запуск дочернего процесса	850	68	745	37
Закрытие cscript	100	6	79	15

Сводка обнаруживает две закономерности, определившие дальнейший анализ.

Во-первых, подавляющая часть событий - обращения к реестру (3378 из 4088 ≈ 83 %). Это прямое следствие архитектуры WSH: почти каждая строка скрипта, вызывающая CreateObject, приводит к поиску CLSID в реестре, чтению InprocServer32 и загрузке соответствующей DLL. Собственно полезная работа (файловые операции, создание процесса) занимает доли процента трассы.

Во-вторых, распределение по группам качественно различно. Простые файловые действия (чтение, удаление) дают короткие трассы, тогда как запуск дочернего процесса порождает более тысячи событий - интерпретатор выполняет проверку политик, AppCert DLLs, SafeBoot, BAM и наследование окружения.

Анализ операций cscript

Запуск cscript и загрузка движка

Сценарий охватывает промежуток от создания процесса до чтения файла скрипта. Зарегистрировано 2650 событий.

Операция	Количество
CloseFile	93
CreateFile	103
CreateFileMapping	64
DeviceIoControl	5
FileSystemControl	12
Load Image	29
QueryBasicInformationFile	33
QueryDirectory	5
QueryNameInformationFile	22
QueryStandardInformationFile	11
ReadFile	47
RegCloseKey	242
RegCreateKey	1
RegEnumKey	91
RegEnumValue	176
RegOpenKey	534
RegQueryKey	748
RegQueryValue	384
RegSetInfoKey	40
Thread Create	7
Thread Exit	1
WriteFile	2

Инициализация процесса. Трасса начинается событием Process Start:
23:50:02,5339754 Process Start cscript.exe (PID 28632)
Parent PID: 7424
Command line: "C:\WINDOWS\system32\cscript.exe" //nologo test.vbs
Current directory: C:\Users\trofi\OneDrive\Desktop\

Сразу за ним следуют Thread Create (главный поток 28372) и серия Load Image: cscript.exe → ntdll.dll → kernel32.dll → KernelBase.dll → msvcrt.dll → oleaut32.dll → ucrtbase.dll → combase.dll → rpcrt4.dll → ole32.dll → gdi32.dll → user32.dll → advapi32.dll → sechost.dll. Загрузка oleaut32.dll, combase.dll,

rpcrt4.dll и ole32.dll отражает подготовку COM-инфраструктуры: без неё вызов CreateObject в скрипте был бы невозможен.

Поиск движка через реестр. Ключевой момент всей трассы - определение языка по расширению файла. Последовательность операций:
23:50:02,5754750 RegOpenKey HKCU\Software\Microsoft\Windows Script Host\Settings
23:50:02,5755266 RegOpenKey HKLM\Software\Microsoft\Windows Script Host\Settings
23:50:02,5755972 RegQueryValue HKLM\...\Settings\Enabled = 1
23:50:02,5902725 RegOpenKey HKCR\.vbs
23:50:02,5903457 RegQueryValue HKCR\.vbs\(Default) = VBSFile
23:50:02,5904591 RegOpenKey HKCR\VBSFile\ScriptEngine
23:50:02,5905490 RegQueryValue HKCR\VBSFile\ScriptEngine\(Default) = VBScript
23:50:02,5912573 RegQueryValue HKCR\VBScript\CLSID\(Default) = {B54F3741-5B07-11cf-A4B0-00AA004A55E8}
23:50:02,5946401 RegQueryValue HKCR\CLSID {B54F3741-...}\InprocServer32\(Default) = C:\Windows\System32\vbscript.dll
23:50:02,5988332 Load Image C:\Windows\System32\vbscript.dll

Это иллюстрирует архитектуру WSH: cscript сам не интерпретирует код. Он читает реестр, чтобы по расширению .vbs найти ProgID VBSFile, по нему - имя движка VBScript, по нему - CLSID {B54F3741-...}, по CLSID - путь к vbscript.dll, и только после этого загружает DLL. Только на 23:50:02,7168170 появляется ReadFile файла test.vbs (Offset 0, Length 704) - чтение самого скрипта.

AMSI. Перед чтением скрипта cscript загружает amsi.dll (23:50:02,6039325) и обращается к провайдерам антивируса:
23:50:02,6056052 RegOpenKey HKLM\Software\Microsoft\AMSI\Providers
23:50:02,6057298 RegEnumKey ...\Providers\Index 0: {3C92AB06-BA68-4DE2-87CA-96570B547BCA}
23:50:02,6060302 RegQueryValue ...\Providers\{3C92AB06-...}\InprocServer32\(Default) = C:\Program Files (x86)\Kaspersky Lab\Kaspersky 21.26\x64\com_antivirus.dll
23:50:02,6115234 CreateFile ...\com_antivirus.dll

Это стандартный механизм Antimalware Scan Interface: содержимое скрипта передаётся антивирусному провайдеру до исполнения.

Итог сценария. Из 2650 событий собственно чтение скрипта - две операции ReadFile. Остальные приходятся на инициализацию: проекцию образов (29 Load Image), поиск движка в реестре (2170 реестровых событий) и создание вспомогательных потоков (7 Thread Create, 1 Thread Exit). Наличие завершившихся потоков при работающем процессе соответствует вспомогательным потокам, создаваемым на время инициализации COM.

Создание и запись файла

Выполняется первой строкой работы с файлами. Зарегистрировано 479 событий:

Операция	Количество
CloseFile	8
CreateFile	9
CreateFileMapping	10
Load Image	5
QueryBasicInformationFile	3
QueryNameInformationFile	5
QueryStandardInformationFile	4
ReadFile	41
RegCloseKey	33
RegOpenKey	110
RegQueryKey	170
RegQueryValue	61
RegSetInfoKey	10
Thread Create	4
Thread Exit	4
WriteFile	2

Загрузка FileSystemObject. Перед первой файловой операцией скрипт вызывает CreateObject("Scripting.FileSystemObject"). Это порождает поиск CLSID по ProgID:
23:50:02,7261817 RegOpenKey HKCR\Scripting.FileSystemObject
23:50:02,7266673 RegQueryValue HKCR\Scripting.FileSystemObject\CLSID\(Default) = {0D43FE01-F093-11CF-8940-00A0C9054228}

23:50:02,7297787 RegQueryValue HKCR\CLSID\{0D43FE01-
...}\InprocServer32\(Default) = C:\Windows\System32\scrrun.dll
23:50:02,7367074 Load Image   C:\Windows\System32\scrrun.dll

Показательно, что одна строка скрипта Set fso = CreateObject(...) обходится в сотни реестровых событий: cscript многократно перебирает ветви HKCU\Software\Classes и HKCR, проверяя TreatAs, InprocServer32, InprocHandler32, LocalServer32 - стандартный алгоритм разрешения COM-класса.

Создание и запись. Собственно файловая работа занимает семь событий:
23:50:02,7632626 CreateFile   C:\Users\trofi\AppData\Local\Temp\cscript_test.txt
                  Desired Access: Generic Write, Read Attributes,
                  Disposition: OverwriteIf, OpenResult: Created
23:50:02,7635513 WriteFile   ...\cscript_test.txt Offset: 0, Length: 19
23:50:02,7637787 WriteFile   ...\cscript_test.txt Offset: 19, Length: 2   (\r\n)
23:50:02,7902901 CloseFile   ...\cscript_test.txt

CreateFile с OpenResult: Created - это и есть создание файла. Два WriteFile - запись строки «Created by cscript!» (19 байт) и перевода строки (2 байта). CloseFile освобождает дескриптор.

Чтение файла

Зарегестрировано 3 события:

Операция	Количество
CloseFile	1
ReadFile	1
CreateFile	1

Эта операция короткая:
23:50:02,7906375 CreateFile C:\Users\trofi\AppData\Local\Temp\cscript_test.txt
                  Desired Access: Generic Read, OpenResult: Opened
23:50:02,7907862 ReadFile   ...\cscript_test.txt Offset: 0, Length: 21
23:50:02,8011578 CloseFile   ...\cscript_test.txt
ReadFile с Length: 21 - это ровно 19 байт строки + 2 байта \r\n. Метод ReadLine в FSO читает строку целиком до разделителя; видно, что библиотека за один вызов запрашивает ровно длину строки.

Удаление файла

Зарегестрировано 6 событий:

Операция	Количество
CreateFile	2
CloseFile	1
SetDispositionInformationEx	1
QueryDirectory	1
QueryAttributeTagFile	1

Метод DeleteFile выполняется в несколько шагов:

```
23:50:02,8104541  CreateFile     C:\Users\trofi\AppData\Local\Temp
                    Options: Directory, OpenResult: Opened
23:50:02,8105350  QueryDirectory ...\Temp\cscript_test.txt
23:50:02,8107546  CreateFile     ...\cscript_test.txt
                    Desired Access: Read Attributes, Delete
23:50:02,8109268  QueryAttributeTagFile  ...\cscript_test.txt  Attributes: A
23:50:02,8109646  SetDispositionInformationEx ...\cscript_test.txt
                    Flags: FILE_DISPOSITION_DELETE,
                        FILE_DISPOSITION_POSIX_SEMANTICS
23:50:02,8110857  CloseFile      ...\cscript_test.txt
```

Ключевой момент - операция SetDispositionInformationEx с флагом FILE_DISPOSITION_DELETE: файл не удаляется отдельным вызовом, а помечается на удаление при закрытии последнего дескриптора. Флаг FILE_DISPOSITION_POSIX_SEMANTICS означает, что имя освобождается немедленно, даже если файл ещё кем-то открыт - семантика, заимствованная из POSIX.

Запуск дочернего процесса cmd

Зарегистрировано 850 событий (68 файловое, 745 реестровых, 37 в группе процессов), из которых собственно создание процесса - одна операция.

Операция	Количество
CloseFile	29
CreateFile	33
CreateFileMapping	2
DeviceIoControl	3
FileSystemControl	4
Process Create	1
QueryBasicInformationFile	9
QueryDirectory	3
QueryNameInformationFile	7
QuerySecurityFile	4
QueryStatInformation	6
RegCloseKey	55
RegEnumValue	4
RegOpenKey	239
RegQueryKey	350
RegQueryValue	92
RegSetInfoKey	4
RegSetValue	1
Thread Create	2
Thread Exit	2

Подготовка запуска. Интерпретатор последовательно проверяет, что cmd.exe существует и может быть запущен:

23:50:03,0848770 CreateFile C:\Windows\System32\cmd.exe
Desired Access: Read Attributes, OpenResult: Opened
23:50:03,0858908 CreateFile C:\Windows\System32\cmd.exe (повторно)
23:50:03,1017044 CreateFile C:\Windows\System32\cmd.exe
Desired Access: Read Data/List Directory,
Execute/Traverse, Synchronize
23:50:03,1018002 CreateFileMapping C:\Windows\System32\cmd.exe
SyncType: SyncTypeCreateSection, PAGE_EXECUTE

Далее - проверка политик безопасности, каждая из которых даёт десятки реестровых событий:

23:50:03,1263577 RegOpenKey
HKLM\System\CurrentControlSet\Control\Session Manager\AppCertDlls
23:50:03,1265082 RegOpenKey
HKLM\System\CurrentControlSet\Control\SafeBoot\Option
23:50:03,1266884 RegOpenKey
HKLM\Software\Policies\Microsoft\Windows\Safer\CodeIdentifiers
23:50:03,1267785 RegQueryValue ...\codeidentifiers\AuthenticodeEnabled = 0
23:50:03,1270955 RegOpenKey
HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\Shell Folders
23:50:03,1271854 RegQueryValue ...\Shell Folders\Cache = ...\INetCache

Особый интерес представляет BAM (Background Activity Moderator) - служба, фиксирующая запуск исполняемых файлов:
23:50:03,1204128 RegOpenKey
HKLM\System\CurrentControlSet\Services\bam\State\UserSettings\S-1-5-21-...
23:50:03,1205028 RegQueryValue
...\...\Device\HarddiskVolume3\Windows\System32\cmd.exe NAME NOT
FOUND
23:50:03,1205492 RegSetValue
...\...\Device\HarddiskVolume3\Windows\System32\cmd.exe Type:
REG_BINARY, Length: 24

Это единственная запись в реестр за весь прогон (не считая служебных записей WSH). Она показывает, что запуск любого .exe неявно модифицирует состояние системы - даже если сам скрипт ничего в реестр не пишет.

Создание процесса. Собственно запуск - одно событие:
23:50:03,1262098 Process Create C:\Windows\System32\cmd.exe
                  PID: 27944
                  Command line: "C:\Windows\System32\cmd.exe" /c echo
Son proccess

До него следуют Thread Create (2) и после него Thread Exit (2) - это потоки самого cscript, ожидающего завершения дочернего процесса (shell.Run ..., 0, True - синхронный запуск с ожиданием).

Почему так много событий. Из 850 событий собственно создание процесса - одно. Остальные 850 приходятся на:

- разрешение пути к cmd.exe;
- проверку политик (AppCertDlls, SafeBoot, Safer\CodeIdentifiers, Image File Execution Options) - большая часть реестровых событий;
- фиксацию запуска в BAM;
- наследование окружения.

Создание дочернего процесса обходится системе существенно дороже любой файловой операции: 850 событий против 3-475 у файловых сценариев.

Закрытие cscript

Зарегистрировано 100 событий:

Операция	Количество
CloseFile	6
Process Exit	1
RegCloseKey	67
RegCreateKey	2
RegOpenKey	1
RegQueryKey	2
RegQueryValue	4
RegSetValue	3
Thread Exit	14

Завершение процесса фиксируется событием Process Exit с Exit Status: 0 - успешное завершение. Пиковый рабочий набор ~30 МБ - типичное значение для cscript, загрузившего vbscript.dll, scrrun.dll, wshom.ocx, combase.dll и антивирусный AMSI-провайдер.

Освобождение ресурсов. После Process Exit в журнале присутствуют закрытие ключа BAM и закрытие дескриптора рабочего каталога C:\Users\trofi\OneDrive\Desktop - он был открыт ещё на этапе инициализации (23:50:02,5469975) и держался всё время работы скрипта.

Показательно, что RegCloseKey для большинства ключей выполняется до Process Exit - в основном на этапе разрешения COM-классов (в трассе сотни пар RegOpenKey/RegCloseKey). К моменту завершения процесса почти все

дескрипторы уже освобождены; система лишь фиксирует факт выхода в BAM.

Сопоставление наблюдений и неожиданные результаты

Действие	Всего	Файлы	Реестр	Процессы	Полезные процессы
Запуск cscript и загрузка движка	2650	324	2170	156	1 (ReadFile)
Создание и запись файла	479	70	384	25	3 (CreateFile и 2 WriteFile)
Чтение файла	3	3	0	0	1 (ReadFile)
Удаление файла	6	5	0	1	1 (SetDispositionInformationEx)
Запуск дочернего процесса	850	68	745	37	1 (Process Create)
Закрытие cscript	100	6	79	15	1 (Process Exit)

Общая закономерность состоит в том, что доля операций, непосредственно выполняющих заявленное действие, ничтожно мала: при создании файла — три операции из 479, при чтении — одна из 479, при запуске процесса — одна из 2650. Основную часть трассы во всех сценариях составляют разрешение COM-классов в реестре, проверка политик безопасности и сбор метаданных.

Первый неожиданный результат — непропорционально большой объём реестровых обращений. На первый взгляд cscript «работает с файлами», но 83% событий — это реестр. Причина в архитектуре WSH: каждая строка CreateObject(...) запускает полный алгоритм разрешения COM-класса (перебор HKCU\Software\Classes, HKCR, TreatAs, InprocServer32, InprocHandler32, LocalServer32), который занимает сотни событий. Сам скрипт при этом не делает ни одной явной записи в реестр — все обращения чтения.

Второй неожиданный результат — присутствие Load Image антивирусной библиотеки com_antivirus.dll (Kaspersky) в трассе cscript. Это не действие скрипта, а работа AMSI: содержимое test.vbs передаётся антивирусному

провайдеру до исполнения, при этом скрипт не обращается к антивирусу, но антивирусная DLL загружается в его адресное пространство.

Вывод

В ходе лабораторной работы с помощью средства диагностики Process Monitor удалось связать действия скрипта cscript с различными системными вызовами и проследить архитектуру Windows Script Host.

Количественное сопоставление показало, что стоимость операций определяется не сложностью команды, а задействованными механизмами операционной системы. Создание дочернего процесса (850 событий) обходится дороже, чем создание, запись, чтение и удаление файла вместе взятые (479 + 3 + 6 = 488 событий). При этом «полезная» работа в каждом сценарии занимает одну-три операции, а остальное — инфраструктура: разрешение COM-классов, проверка политик, сбор метаданных.

Архитектурный результат: трасса наглядно показывает двухступенчатую схему WSH. cscript сам не интерпретирует код — он по расширению .vbs находит в реестре ProgID VBSFile, по нему — движок VBScript, по нему — CLSID, по CLSID — путь к vbscript.dll, и только после загрузки DLL читает файл скрипта. Аналогично каждая строка CreateObject(...) запускает полный алгоритм разрешения COM-класса, что даёт сотни реестровых событий на одну строку кода.

Практическим результатом является восстановление логики работы программы по контексту: последовательности операций, путям, результатам и связи между разнесёнными во времени сценариями. По трассе однозначно восстанавливается, что скрипт создал файл, записал в него строку, прочитал её, удалил файл и запустил дочерний процесс — при том, что сам текст скрипта в трассе не фигурирует.

Отдельным результатом следует считать выявленные неожиданные эффекты: непропорциональный объём реестровых обращений (83 %), загрузку антивирусной DLL через AMSI.

Приложения

Приложение А: скрипт test.vbs

```
Dim fso, shell, file
Set fso   = CreateObject("Scripting.FileSystemObject")
Set shell = CreateObject("WScript.Shell")

Dim testFile
testFile = shell.ExpandEnvironmentStrings("%TEMP%") & "\cscript_test.txt"

Set file = fso.CreateTextFile(testFile, True)
file.WriteLine "Created by cscript!"
file.Close

Set file = fso.OpenTextFile(testFile, 1)
WScript.Echo "Readed: " & file.ReadLine
file.Close

fso.DeleteFile testFile, True

shell.Run "cmd /c echo Son proccess", 0, True

WScript.Echo "End."
```

Приложение Б: файлы с логами ProcMon

https://github.com/Tr0f1mka/os_lab1/blob/main/Logfile.CSV
https://github.com/Tr0f1mka/os_lab1/blob/main/Logfile.PML